Univerzitet u Sarajevu
Elektrotehnički fakultet
Predmet: Mobilna robotika
Predmetni profesor: prof. dr Velagić Jasmin
Akademska godina: 2025/2026.

Laboratorijska vježba 4

Prvi dio vježbe

Za prvi dio vježbe potrebno je instalirati SLAM toolbox:

```
sudo apt install ros-kilted-slam-toolbox
```

Moguće je da na vašoj mašini je već instaliran SLAM toolbox, ali na ovaj način možemo to i provjeriti.

Potrebno je kreirati novi paket unutar našeg workspacea u `src` folderu pod nazivom `vjezba4`. Onda je potrebno uraditi sljedeće kako bismo kreirali launch file:

```
cd ime_studenta_ws/src/vjezba4
mkdir launch
cd launch
nano moj_mapping.launch.py
```

Unutar file-a `moj_mapping.launch.py` pasteati sljedeći kod:

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, EmitEvent
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import LifecycleNode
from lifecycle_msgs.msg import Transition
from launch_ros.events.lifecycle import ChangeState


def generate_launch_description():
    use_sim_time = LaunchConfiguration('use_sim_time', default='true')

    # --- Define the SLAM Toolbox lifecycle node ---
    slam_toolbox_node = LifecycleNode(
        package='slam_toolbox',
        executable='async_slam_toolbox_node',
        name='slam_toolbox',
        namespace='',
        output='screen',
        parameters=[{
            'use_sim_time': use_sim_time,
            'odom_frame': 'odom',
            'base_frame': 'base_footprint',
            'scan_topic': '/scan',
            'slam_params_file': '/opt/ros/kilted/share/slam_toolbox/config/
               mapper_params_online_async.yaml'
        }],
    )

    # --- Emit lifecycle events manually ---
    configure = EmitEvent(
        event=ChangeState(
            lifecycle_node_matcher=lambda n: n == slam_toolbox_node,
            transition_id=Transition.TRANSITION_CONFIGURE,
        )
```

```
    )
    activate = EmitEvent(
        event=ChangeState(
            lifecycle_node_matcher=lambda n: n == slam_toolbox_node,
            transition_id=Transition.TRANSITION_ACTIVATE,
        )
    )

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_sim_time',
            default_value='true',
            description='Use simulation (Gazebo) clock if true'
        ),
        slam_toolbox_node,
        configure,
        activate,
    ])
```

Potrebno je onda buildati paket. Nakon toga, otvoriti četiri terminala i upisati sljedeće:
Terminal 1:

```
cd ime_studenta_ws
source /opt/ros/kilted/setup.bash
source install/setup.bash
ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py
```

Terminal 2:

```
cd ime_studenta_ws
source /opt/ros/kilted/setup.bash
source install/setup.bash
ros2 launch turtlebot3_bringup rviz2.launch.py
```

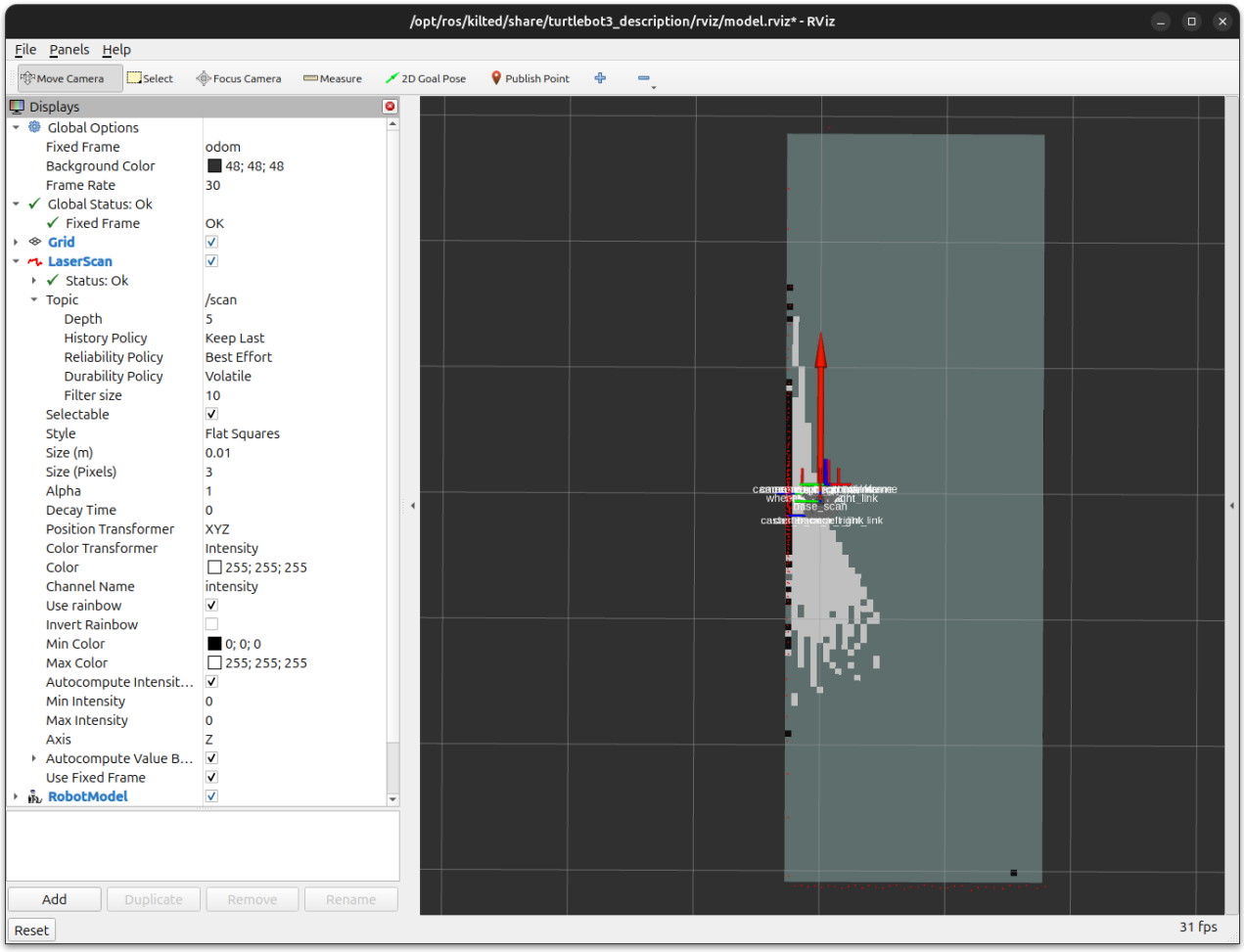
Terminal 3:

```
cd ime_studenta_ws
source /opt/ros/kilted/setup.bash
source install/setup.bash
ros2 run turtlebot3_teleop teleop_keyboard
```

Terminal 4:

```
cd ime_studenta_ws
source /opt/ros/kilted/setup.bash
source install/setup.bash
ros2 launch vjezba4 moj_mapping.launch.py
```

Nakon toga u RVizu je potrebno dodati topic Map nakon čega ćete dobiti nešto slično kao na slici 1.



Slika 1: RViz nakon dodavanja Map teme

Pokretanjem robota korištenjem teleopa ćete dobiti mapiranje prostora po kome se robot kretao.

Drugi dio vježbe

Potrebno je u terminalu pokrenuti komande za instaliranje sljedećih paketa:

```
sudo apt install ros-kilted-octomap-server
sudo apt install ros-kilted-octomap-msgs
sudo apt install ros-kilted-octomap-ros
```

Da biste provjerili jesu li se paketi ispravno instalirali, pokrenite naredbu: `ros2 pkg list | grep octomap` nakon koje bi trebao izlaz biti sljedeći:

```
octomap_msgs
octomap_ros
octomap_server
```

Trebamo sada novi launch file kreirati u paketu `vjezba4`:

```
cd ime_studenta_ws/src/vjezba4/launch
nano moj_octomap.launch.py
```

Unutar launch filea pasteati sljedeći kod:

```
from launch import LaunchDescription
from launch_ros.actions import Node
```

```
def generate_launch_description():
    return LaunchDescription([
        # 1) Convert /scan (LaserScan) to /scan_cloud (PointCloud2)
        Node(
            package='vjezba4',
            executable='scan_to_cloud',
            name='scan_to_cloud',
            output='screen'
        ),

        # 2) Run Octomap server on /scan_cloud
        Node(
            package='octomap_server',
            executable='octomap_server_node',
            name='octomap_server',
            output='screen',
            parameters=[{
                'frame_id': 'odom',
                'resolution': 0.05,
                'sensor_model/max_range': 5.0,
            }],
            remappings=[
                ('cloud_in', '/scan_cloud'),
            ]
        )
    ])
```

Nakon toga je potrebno i editovati `package.xml`:

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
   schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>vjezba4</name>
  <version>0.0.0</version>
  <description>ROS 2 mapping and octomap exercise</description>
  <maintainer email="anniberi@todo.todo">anniberi</maintainer>
  <license>Apache-2.0</license>

  <!-- Build tool -->
  <buildtool_depend>ament_cmake</buildtool_depend>

  <!-- Runtime dependencies -->
  <depend>rclcpp</depend>
  <depend>sensor_msgs</depend>
  <depend>octomap_server</depend>
  <depend>octomap_msgs</depend>
  <depend>octomap_ros</depend>

  <!-- Testing -->
  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

`CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.20)
```

```
project(vjezba4)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
# uncomment the following section in order to fill in
# further dependencies manually.
# find_package(<dependency> REQUIRED)

find_package(laser_geometry REQUIRED)

install(
  DIRECTORY launch
  DESTINATION share/${PROJECT_NAME}
)

install(PROGRAMS
  scan_to_cloud.py
  DESTINATION lib/${PROJECT_NAME}
  RENAME scan_to_cloud
)


if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  # the following line skips the linter which checks for copyrights
  # comment the line when a copyright and license is added to all source
     files
  set(ament_cmake_copyright_FOUND TRUE)
  # the following line skips cpplint (only works in a git repo)
  # comment the line when this package is in a git repo and when
  # a copyright and license is added to all source files
  set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()
```

Pored svega toga potrebno je unutar foldera `vjezba4` napraviti file `scan_to_cloud.py`:

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan, PointCloud2
import laser_geometry.laser_geometry as lg


class ScanToCloud(Node):
    def __init__(self):
        super().__init__('scan_to_cloud')
        self.lp = lg.LaserProjection()
        self.sub = self.create_subscription(LaserScan, '/scan', self.
           callback, 10)
        self.pub = self.create_publisher(PointCloud2, '/scan_cloud', 10)
        self.get_logger().info('Scan-to-Cloud converter running...')

    def callback(self, msg):
        try:
            cloud = self.lp.projectLaser(msg)
```

```
            self.pub.publish(cloud)
        except Exception as e:
            self.get_logger().warn(f"Conversion failed: {e}")


def main(args=None):
    rclpy.init(args=args)
    node = ScanToCloud()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Nakon toga, potrebno je sve buildati i sourceati. Otvoriti tri terminala i upisati sljedeće komande:

Terminal 1:

```
cd ime_studenta_ws
source install/setup.bash
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Terminal 2:

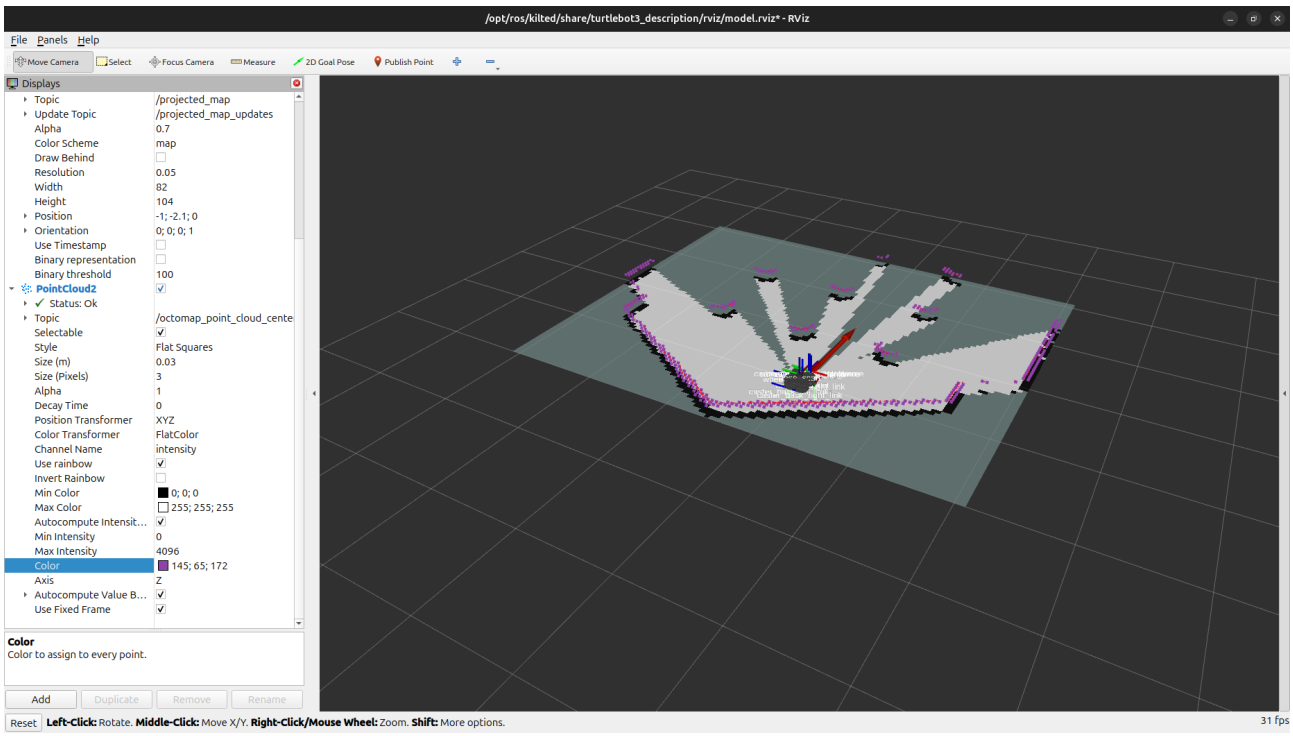
```
cd ime_studenta_ws
source install/setup.bash
ros2 launch vjezba4 moj_octomap.launch.py
```

Terminal 3:

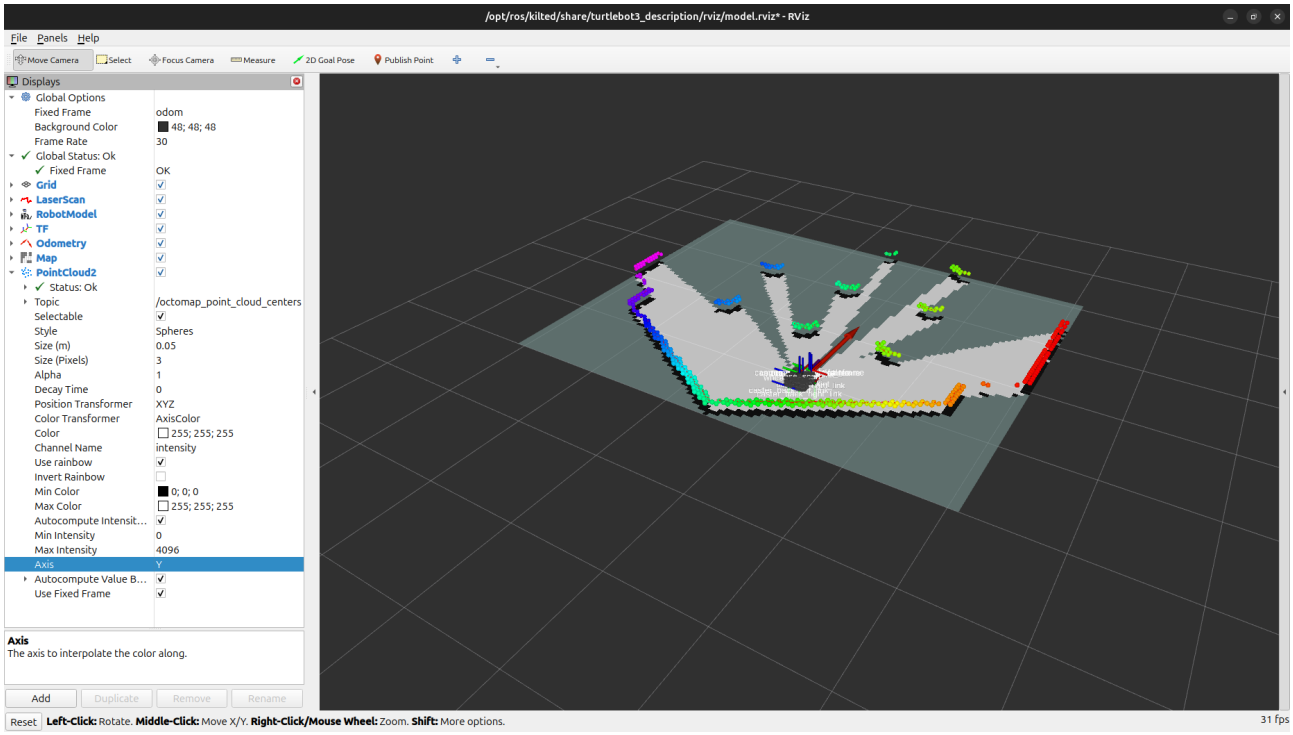
```
cd ime_studenta_ws
source install/setup.bash
ros2 launch turtlebot3_bringup rviz2.launch.py
```

Unutar RViz-a, potrebno je otići na `Add - By Topic - PointCloud2`. Unutar polja za topic selektovati `\octomap_point_cloud_centers`, Color Transformer postaviti na `FlatColor` i u polju za Color odabrati boju po želji. Nakon toga, ako želite možete dodati i topic `Map - projected_map` ako želite da imate shading kao što je prikazan na slici 2.

Moguće je i unutar PointCloud2 odabrati x ili y osu ako se izabere Color Transformer `AxisColor`, pa da se dobije prikaz mapiranja u obliku duge na osnovu toga šta je bliže a šta dalje robotu po x, odnosno y, osi, kao što je prikazano na slici 3.



Slika 2: RViz izgled za octomapping



Slika 3: RViz prikaz za octomapping sa dugom